SafePal TRXStaking

Smart Contract Security Audit Report

Audited by: TenArmor

Audit dates: Mar 5, 2026 – Mar 10, 2026

Table of Contents

1. Introduction

1.1 About TenArmor

TenArmor is a leading Web3 security firm dedicated to safeguarding blockchain ecosystems. We offer cutting-edge security solutions tailored to the complexities of decentralized technologies. With our flagship products, TenMonitor for real-time threat detection and response and TenTrace for a one-stop asset recovery solution, we provide an all-encompassing approach to Web3 security.

Learn more about us:

Official Website: TenArmor.com

X: @TenArmor, @TenArmorAlert

mail: team@tenarmor.com

TenMonitor: TenMonitor.com

TenTrace: TenTrace.com

1.2 Disclaimer

This report represents an analysis performed within a defined scope and time frame, based on the materials and documentation provided. It is not exhaustive and does not encompass all potential vulnerabilities.

A smart contract security review aims to identify as many vulnerabilities as possible within the given constraints of time, resources, and expertise. However, it cannot guarantee the complete absence of vulnerabilities or the complete security of the project. This report does not serve as an endorsement of the underlying business or product and was limited to an evaluation of the security aspects of the Solidity implementation of the contracts.

To enhance security, we strongly recommend conducting subsequent security reviews, implementing bug bounty programs, and maintaining ongoing on-chain monitoring.

1.3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low

Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2. Executive Summary

2.1 About SafePal TRXStaking

TRXStaking is a custodial, share-based TRX staking vault deployed on the TRON mainnet. Users deposit TRX to receive non-transferable internal shares; redemption is through a delayed withdrawal mechanism (15-day lock period). Yield is governed by an admin-configured APR using a linear (simple-interest) accrual model. Staked TRX is immediately forwarded to an operator-managed hot wallet for off-chain yield strategies (e.g., TRON Stake 2.0). The contract maintains an internal accounting variable (totalReserve) representing the theoretical NAV, rather than relying on the contract's actual TRX balance. This architectural choice provides natural immunity against classic vault inflation attacks (ERC-4626 donation-style attacks).

2.2 Audit Scope

File: stakeTrx.sol

Solidity Version: ^0.8.20

Dependencies: OpenZeppelin Contracts 5.x (Ownable, Pausable, ReentrancyGuard)

2.3 Findings Count

Severity	Count
Critical	0
High	0
Medium	0
Low	0
Informational	5
Total Findings	5

3. Findings Summary

ID	Description	Status
[I–1]	Operational risk: misconfigured hot wallet can cause DoS on stake functionality.	Acknowledged
[I–2]	setApr allows setting the same APR value.	Acknowledged
[I–3]	No on–chain enumeration of user withdrawal requests.	Acknowledged
[I–4]	pendingHotWallet overwrite does not emit explicit cancellation event.	Acknowledged
[I–5]	validShares modifier contains a redundant check (gas optimization).	Acknowledged

4. Findings

4.1 Informational

[I-1] Operational risk: misconfigured hot wallet can cause DoS on stake functionality

Description:

In stake(), staked TRX is immediately forwarded to hotWallet via hotWallet.call{value: msg.value}(""). If the hot wallet is set to a contract address that reverts on receive (or an otherwise non-payable address), all stake() calls will revert. Similarly, emergencyWithdraw() sends funds to hotWallet and would also be blocked.

stakeTrx.sol Line#149

```
123     function stake()
124         external
125         payable
126         nonReentrant
127         whenNotPaused
128         returns (uint256 shares)
129     {
130         require(msg.value >= MIN_STAKE_AMOUNT, "TRXStaking: amount too low");
131         require(msg.value <= MAX_STAKE_AMOUNT, "TRXStaking: amount too high");
132 
133         _updateReserve();
134 
135         uint256 currentTotalShares = totalShares;
136         if (currentTotalShares == 0) {
137             shares = msg.value;
138         } else {
139             shares = _getSharesByTrx(msg.value);
140             require(shares > 0, "TRXStaking: stake too small");
141         }
142 
143         uint256 oldUserShares = userShares[msg.sender];
144         userShares[msg.sender] = oldUserShares + shares;
145         totalShares = currentTotalShares + shares;
146         totalReserve += msg.value;
147         emit SharesChanged(msg.sender, oldUserShares, userShares[msg.sender], 1);
148 
149         (bool sent, ) = hotWallet.call{value: msg.value}("");
150         require(sent, "TRXStaking: transfer to hot wallet failed");
151 
152         emit Staked(msg.sender, msg.value, shares);
153         return shares;
154     }
```

stakeTrx.sol Line#270

```
266     function emergencyWithdraw() external onlyOwner nonReentrant {
267         uint256 balance = address(this).balance;
268         require(balance > 0, "TRXStaking: no balance");
269 
270         (bool sent, ) = hotWallet.call{value: balance}("");
271         require(sent, "TRXStaking: transfer failed");
272 
273         emit EmergencyWithdraw(balance);
274     }
```

Impact:

Staking functionality would be blocked until the hot wallet is corrected via setHotWallet + acceptHotWallet (2-day timelock). During the pending period, the old (working) hot wallet remains active, so existing operations are not disrupted. claimWithdraw is unaffected as it transfers to msg.sender, not the hot wallet. This is a purely operational/configuration risk.

Recommendation:

Before calling acceptHotWallet(), operators should verify off-chain that the pending address can receive native TRX (e.g., send a small test transaction from the hot wallet). Document this verification step in the operational playbook. The 2-day timelock already provides a sufficient correction window.

[I-2] setApr allows setting the same APR value

Description:

The setApr function does not check whether newApr equals the current apr. The owner can call setApr(apr) which wastes gas and emits a misleading APRUpdated event with identical old and new values.

stakeTrx.sol Line#281

```
281     function setApr(uint256 newApr) external onlyOwner {
282         require(newApr <= MAX_APR, "TRXStaking: exceeds max");
283         _updateReserve();
284         uint256 oldApr = apr;
285         apr = newApr;
286         emit APRUpdated(oldApr, newApr);
287     }
```

Impact:

No security impact. Potentially confusing for off-chain monitoring systems that treat every APRUpdated event as a meaningful parameter change.

Recommendation:

Add require(newApr != apr, "TRXStaking: APR same as current") at the beginning of setApr.

[I-3] No on-chain enumeration of user withdrawal requests

Description:

withdrawRequests uses mapping(address => mapping(uint256 => WithdrawRequest)) with a global auto-incrementing nextRequestId. There is no on-chain mechanism to list all pending withdrawal requests for a given user address.

stakeTrx.sol Line#68-69

```
68  uint256 public nextRequestId;                  // auto-increment ID for withdrawal requests
69  mapping(address => mapping(uint256 => WithdrawRequest)) public withdrawRequests;
```

Impact:

Users must track their requestIds off-chain (e.g., by indexing WithdrawRequested events). This is a UX inconvenience issue.

Recommendation:

Consider maintaining a per-user array of active request IDs (e.g., mapping(address => uint256[]) userRequestIds), or provide a batch query function that checks a range of IDs for a given user.

[I-4] pendingHotWallet overwrite does not emit explicit cancellation event

Description:

When setHotWallet is called while a pendingHotWallet already exists (with a different new address), the old pending address is silently overwritten without emitting a HotWalletUpdateCancelled event for the replaced address. Only a new HotWalletUpdateScheduled event is emitted.

stakeTrx.sol Line#222

```
222  function setHotWallet(address _newHotWallet) external onlyOwner {
223      require(_newHotWallet != address(0), "TRXStaking: zero address");
224      require(_newHotWallet != address(this), "TRXStaking: cannot be self");
225      require(_newHotWallet != hotWallet, "TRXStaking: same as current");
226      require(_newHotWallet != pendingHotWallet, "TRXStaking: already pending");
227
228      pendingHotWallet = _newHotWallet;
229      pendingHotWalletTimestamp = block.timestamp + TIMELOCK_DELAY;
230
231      emit HotWalletUpdateScheduled(hotWallet, _newHotWallet, pendingHotWalletTimestamp);
232  }
```

Impact:

Monitoring systems that rely solely on HotWalletUpdateCancelled events to detect cancellations will miss implicit cancellations caused by overwrites. The information is still inferrable from consecutive HotWalletUpdateScheduled events, but requires additional logic.

Recommendation:

Consider emitting HotWalletUpdateCancelled(pendingHotWallet) before overwriting when pendingHotWallet != address(0). This makes the event stream self-describing without changing the functional behavior.

[I-5] validShares modifier contains a redundant check (gas optimization)

Description:

The validShares modifier checks both shares > 0 and shares <= totalShares. The second condition is redundant because requestWithdraw (the only consumer of this modifier) subsequently requires userShares[msg.sender] >= shares, and

userShares[msg.sender] <= totalShares is a protocol invariant (guaranteed by the share accounting logic in stake and requestWithdraw).

stakeTrx.sol Line#88

```
88  modifier validShares(uint256 shares) {
89      require(shares > 0, "TRXStaking: zero shares");
90      require(shares <= totalShares, "TRXStaking: exceeds total shares");
91      _;
92  }
```

Impact:

Minor unnecessary gas consumption on every requestWithdraw call. No security impact.

Recommendation:

Remove the require(shares <= totalShares, ...) line from the validShares modifier to save gas. The shares > 0 check can optionally be inlined directly into requestWithdraw if the modifier is no longer needed elsewhere.